GeantPy Implementation Plan

Current State

Current pipeline:

```
YAML Configuration
        ↓
Python Interface
        ↓
Generate Geant4 Macro
        ↓
Run Geant4
        ↓
ROOT Output
```

Current limitations:

- Hardcoded particle-target collision application.
- YAML is the primary simulation interface.
- Geometry is not constructed through Python objects.
- Execution pipeline is tightly coupled to the current application.

Phase 1 - Wrap the Existing Pipeline

Goal: Preserve existing functionality while introducing a Python API.

Tasks:

- Create `Simulation` class.
- Create `Beam`, `Target`, and `World` classes.
- Convert Python objects into the existing YAML schema.
- Keep macro generation unchanged.
- Return a `SimulationResults` object.

Acceptance: - Existing simulations run entirely from Python.

Phase 2 - Remove YAML as the User Interface

Tasks:

- Make YAML an internal serialization format.
- Validate Python objects before serialization.
- Hide configuration files from end users.

Acceptance: - Users never edit YAML manually.

Phase 3 - Generalize the World

Tasks:

- Abstract geometry primitives (Box, Cylinder, Sphere).
- Support arbitrary materials.
- Add multiple volumes.
- Add particle sources and physics configuration.

Acceptance: - Particle-target collisions become one specialization of a generic world.

Phase 4 - Backend Refactor

Tasks:

- Introduce a `Backend` interface.
- Move macro generation into `MacroBackend`.
- Isolate executable launching.
- Isolate ROOT parsing.

Acceptance: - Execution pipeline is modular and independently testable.

Phase 5 - Results API

Tasks:

- Parse ROOT into Python classes.
- Support NumPy/Pandas export.
- Add event summaries and helper methods.

Acceptance: - Users rarely interact with ROOT directly.

Phase 6 - Future Extensions

Potential work:

- Native pybind11 backend.
- Remote/HPC execution backend. -
- Parallel parameter sweeps. -
- Plugin architecture for detectors and physics lists.